

PROJECT PRESENTATION

CarRental

A secure fleet and booking
management application

Product overview • User journeys • Architecture • Data • API • Security

Presented by

Karan Nale • Om Bangar • Sanket Narsale
Nishant Khatri • Chirag Dayal

Based on the current CarRental solution in this workspace



What the application does

A lightweight web application for managing a rental fleet and customer reservations.

01

Browse fleet

View vehicles, availability, and daily rates.

02

Manage fleet

Admins add, edit, delete, and mark cars available or unavailable.

03

Book dates

Members reserve a car for a date range with overlap checks.

04

Track bookings

Members see their reservations and may cancel before the start date.

Two roles, distinct responsibilities

Role-based access separates customer actions from fleet administration.

MEMBER

Renter / customer

- Register with a unique email and sign in
- Browse fleet and daily rates
- Create a reservation for selected dates
- View own booking history and cancel eligible bookings

ADMIN

Fleet manager

- All member fleet-browsing capabilities
- Create, update, and delete vehicle records
- Change vehicle availability
- Inspect a read-only database table viewer
- May cancel any booking

Reservation journey

The interface guides a member from account creation through booking management.

1

Create account

Email and strong password; account receives the Member role.

2

Browse cars

See year, make, model, availability, and rate per day.

3

Choose dates

Date picker disallows past dates and reversed ranges.

4

Confirm booking

Server validates vehicle and date overlap; total is calculated.

5

Manage booking

Review details; cancel before the booking start date.

User-scoped booking list: renters retrieve only their own bookings; ownership is checked again on cancellation.

Admin fleet-management workflow

Fleet changes are exposed in the UI and enforced by protected API endpoints.

Vehicle record

- Identification: make, model, year
- Pricing: daily rate (decimal)
- Operational state: available / unavailable

Vehicles begin available by default.

- **Add:** create a car after field validation
- **Edit:** update make, model, year, or rate
- **Toggle:** explicitly set availability
- **Delete:** blocked if booking history exists

Booking, pricing & cancellation rules

Business rules are checked server-side; browser date controls are only a convenience.

DATE RANGE

Start date cannot be in the past; end date must be on or after start.

DURATION

A booking may cover at most 60 days.

AVAILABILITY

Car must exist and be marked available.

NO DOUBLE BOOKING

Confirmed reservations for the same car may not overlap.

TOTAL PRICE

Daily rate × inclusive number of calendar days.

CANCELLATION

Member may cancel only before start; Admin can cancel any booking.

Simple, layered application architecture

One ASP.NET Core web host serves MVC pages and JSON endpoints; EF Core persists data in SQLite.



**Authentication is ASP.NET Core Identity. RentalDbContext extends IdentityDbContext and maps the domain entities.
OpenAPI document is mapped only in Development.**

Core data model & relationships

A booking links one car and one Identity user; deletion rules preserve reservation history.

FLEET ENTITY

Car

- Id (primary key)
- Make + Model (required, max 80 chars)
- Year
- DailyRate (decimal, precision 10,2)
- IsAvailable (default true)

RESERVATION ENTITY

Booking

- Id; CarId + UserId (foreign keys)
- StartDate + EndDate (DateOnly)
- TotalPrice (decimal, precision 10,2)
- Status (default Confirmed; max 20 chars)
- CreatedAtUtc (UTC timestamp)

HTTP API surface

All API calls require authentication; role policies guard write operations.

METHOD	ROUTE	ACCESS	PURPOSE
GET	/api/cars	Authenticated	List fleet
GET	/api/cars/{id}	Authenticated	Get one car
POST	/api/cars	Admin	Create car
PUT	/api/cars/{id}	Admin	Edit car
DELETE	/api/cars/{id}	Admin	Delete car (no booking history)
PUT	/api/cars/{id}/availability	Admin	Set availability
GET	/api/bookings/mine	Member	List current user's bookings
POST	/api/cars/{id}/bookings	Member	Create booking
DELETE	/api/bookings/{id}	Owner or Admin	Cancel booking

Expected outcomes include 200/201, 204, 400, 401/403, 404, and 409 based on operation and validation.

Identity, authorization & defensive controls

The application has baseline controls; production hardening remains necessary.

CONTROL	Identity — ASP.NET Core Identity + EF store; unique email; Admin and Member roles.
Password	12+ characters with uppercase/lowercase, digit, and symbol requirements.
Logout	Lock after 5 failed attempts for 15 minutes.
Cookies	HttpOnly; SameSite=Strict; SecurePolicy=SameAsRequest.
Authorization	Fallback requires an authenticated user; FleetManagers=Admin; Renters=Member.
Request safety	Anti-forgery tokens on account forms; return URL restricted to local URLs.
API behavior	Unauthenticated API requests receive 401; forbidden API requests receive 403.

Important: A demo Admin account is bootstrapped from source code. Replace/remove demo credentials and configure secrets before deployment.

A focused, role-aware web experience

Razor-rendered pages and small client-side scripts keep the demo simple and direct.

Main fleet page

- Fleet cards show vehicle identity, daily rate, and state
- Admins see add/edit/delete and availability controls
- Members see date inputs and a Book action
- Members see their booking list and cancellation action
- Mobile breakpoint stacks page panels and car cards

Account & admin pages

- Dedicated sign-in and registration forms
- Inline model validation messages
- Logout uses a protected POST form
- Admin-only database viewer: table selector, read-only, max 200 rows
- HTML values rendered via Razor encoding

Validation & expected failure paths

Invalid requests are rejected before persistence and common conflicts are explicit.

Car input

- Make/model required; trimmed
- Year ≥ 1886 and $\leq$ current year + 1
- Daily rate > 0

Booking input

- Start date not in past
- End date on/after start; max 60 days
- Car exists, available, and dates do not overlap

Safe operations

- Delete blocked if any booking exists
- Only owner or Admin can cancel
- Member cancellation blocked on/after start

Technology stack & solution layout

The current CarRental.slnx contains two projects and uses modern .NET conventions.

WEB

ASP.NET Core MVC + Minimal APIs

.NET 10 target

AUTH

ASP.NET Core Identity

EF Core store

DATA

EF Core + SQLite

Local database file

DOMAIN

CarRental.Core

Car + Booking models

Local runbook and persistence

Suitable for a local demo; treat database/bootstrap setup as areas to harden for production.

RUN LOCALLY

```
dotnet run --project  
CarRental.Api/CarRental.Api.csproj
```

Run from the CarRental solution folder. Use the local URL printed by ASP.NET Core.

Development OpenAPI: /openapi/v1.json

Database behavior

- Connection string key: RentalDatabase
- Default fallback: Data Source=carrental-secure.db
- EF EnsureCreated at startup; SQL creates booking table/index
- Admin and Member roles seeded at startup
- Demo Admin user and sample cars seeded if missing

Implemented scope vs. production extensions

The left column reflects current source; the right column lists opportunities, not existing features.

IMPLEMENTED

Core workflow

- Identity accounts and Admin/Member authorization
- Fleet CRUD + availability controls
- Booking create/list/cancel with date conflict logic
- SQLite persistence and read-only admin DB viewer
- Development-only OpenAPI mapping and MVC interface

POTENTIAL EXTENSIONS

Production / business additions

- Payment processing, invoices, taxes, deposits
- Pickup/return inspections, damage, maintenance
- Search/filter/sort, vehicle photos, locations
- Email/SMS confirmations and password-recovery UI
- Automated tests, monitoring, deployment pipeline

Project modules and what each one does

The code is separated into focused areas so pages, rules, and database work have clear responsibilities.

USER INTERFACE

Views

Home displays fleet and bookings. **Account** provides login/register. **Database** is an admin-only read-only viewer.

ACCOUNT & ACCESS

Identity

AccountController signs in, registers, assigns Member, and signs out. Authorization policies separate Member and Admin.

HTTP/API

Endpoints

Program.cs maps JSON routes for cars and bookings, checks input, enforces roles, and returns HTTP responses.

BUSINESS DATA

Core models

Car describes a vehicle. **Booking** describes a reservation. Request records carry data into endpoints.

DATA ACCESS

EF Core + SQLite

RentalDbContext maps models to tables. Entity Framework Core translates C# queries into SQL.

DATABASE SETUP

Startup + schema

Startup ensures tables and roles exist, creates the booking index, and inserts the demo admin and sample cars.

End-to-end project workflow

From sign-in to a completed reservation—or from admin login to a managed fleet.

MEMBER FLOW

Customer booking

1. Member registers or signs in; Identity establishes a secure cookie.
2. Browser loads the fleet from `GET /api/cars`.
3. Member chooses a car and start/end dates.
4. Booking endpoint checks dates, maximum duration, car status, and overlap.
5. Valid booking is saved; total price and confirmation are returned.
6. Member views bookings or cancels before the start date.

ADMIN FLOW

Fleet management

1. Admin signs in with the Admin role.
2. Admin adds a car or edits its make, model, year, and daily rate.
3. Admin changes availability or deletes a car with no booking history.
4. API checks role and data, then EF Core saves to SQLite.
5. Updated fleet is returned to the browser and re-displayed.
6. Admin can inspect database tables in the read-only viewer.

How to read the code

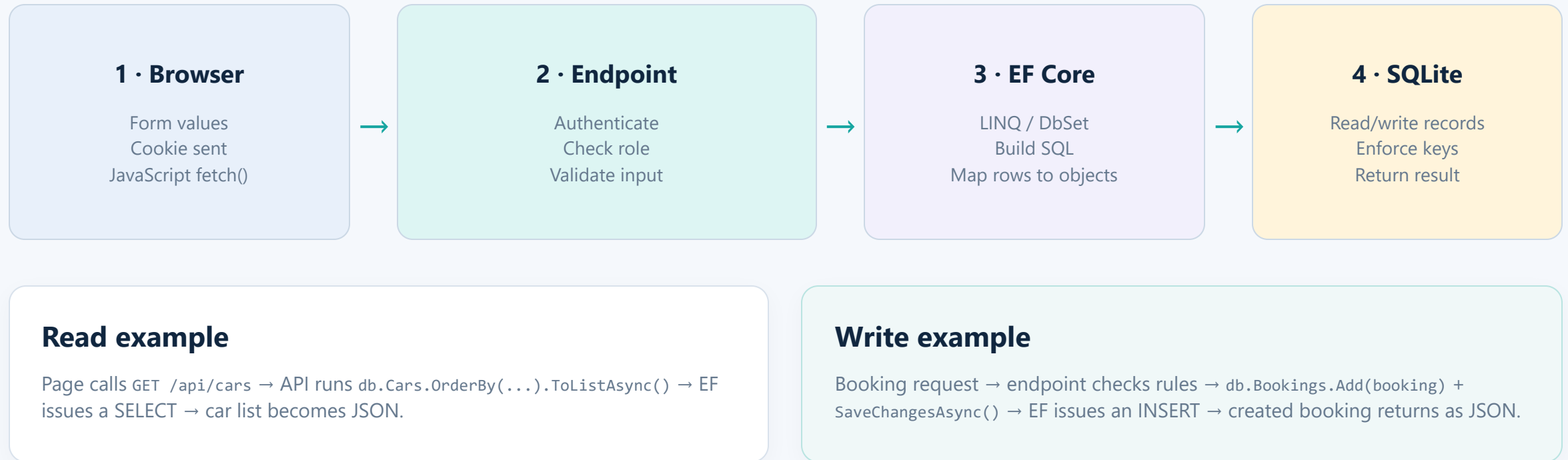
Each file has one main job; follow a request from the web page down to saved data.

FILE / MODULE	IN PLAIN WORDS	EXAMPLE RESPONSIBILITY
Program.cs	Starts the application and connects its parts.	Configure SQLite, Identity, roles, and API routes.
AccountController.cs	Handles account forms and sign-in/out actions.	Create user and give the new renter the Member role.
RentalDbContext.cs	Explains how C# objects are stored as database tables.	Maps Car and Booking; sets keys, relationships, and index.
Car.cs / Booking.cs	Describe the information each record holds.	Car stores price and availability; Booking stores dates and user.
Home/Index.cshtml	Builds the page the user sees and handles button actions.	JavaScript calls JSON endpoints using fetch().
DatabaseController.cs	Lets Admin safely view table names, columns, and up to 200 rows.	Reads metadata and table rows; does not edit them.

Simple example: “Book car” sends car ID + dates → server checks the rules → database saves a Booking row → server sends the booking back to the page.

How data moves through the application

The browser never connects directly to SQLite; all reads and writes pass through protected server code.



Database tables, keys, and important fields

Identity supplies the account tables; the application adds Cars and Bookings.

CARS

Fleet table

FIELD	KEY / RULE	MEANING
Id	PK, auto number	Unique car ID
Make	Required, ≤80	Manufacturer
Model	Required, ≤80	Car model
Year	Required integer	Model year
DailyRate	> 0, decimal	Rental cost/day
IsAvailable	Default 1/true	Operational flag

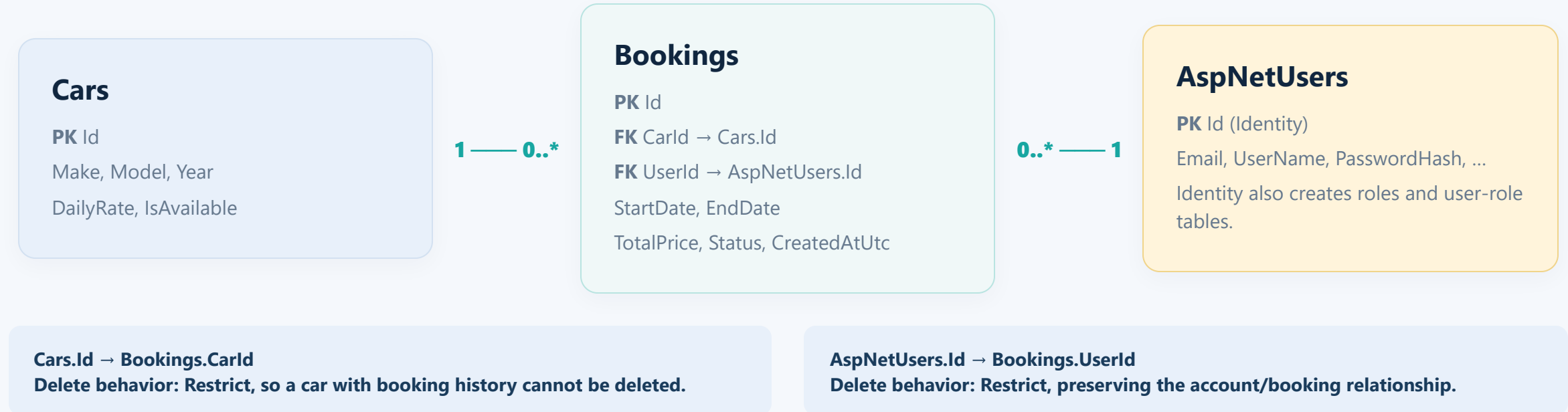
BOOKINGS

Reservation table

FIELD	KEY / RULE	MEANING
Id	PK, auto number	Unique booking ID
CarId	FK → Cars.Id	Booked vehicle
UserId	FK →AspNetUsers.Id	Renter account
StartDate / EndDate	Required dates	Rental date range
TotalPrice	Decimal	Rate × inclusive days
Status / CreatedAtUtc	Defaults / timestamp	Booking state / created time

Entity–relationship diagram

One car can appear in many bookings; one Identity user can also own many bookings.



The database has an index on (CarId, StartDate, EndDate) to support date-conflict lookups. Identity tables are generated by ASP.NET Core Identity, not listed in schema.sql.

SQL used to create the application tables

schema.sql defines Cars, Bookings, foreign keys, checks, defaults, and a booking lookup index.

CREATE FLEET TABLE

```
CREATE TABLE IF NOT EXISTS Cars (  
  Id INTEGER PRIMARY KEY AUTOINCREMENT,  
  Make TEXT NOT NULL CHECK(length(Make) <= 80),  
  Model TEXT NOT NULL CHECK(length(Model) <= 80),  
  Year INTEGER NOT NULL,  
  DailyRate NUMERIC NOT NULL CHECK(DailyRate > 0),  
  IsAvailable INTEGER NOT NULL DEFAULT 1  
);
```

CREATE BOOKINGS + INDEX

```
CREATE TABLE IF NOT EXISTS Bookings (  
  Id INTEGER PRIMARY KEY AUTOINCREMENT,  
  CarId INTEGER NOT NULL,  
  UserId TEXT NOT NULL,  
  StartDate TEXT NOT NULL, EndDate TEXT NOT NULL,  
  TotalPrice NUMERIC NOT NULL,  
  Status TEXT NOT NULL DEFAULT 'Confirmed',  
  CreatedAtUtc TEXT NOT NULL,  
  FOREIGN KEY (CarId) REFERENCES Cars(Id)  
    ON DELETE RESTRICT,  
  FOREIGN KEY (UserId) REFERENCESAspNetUsers(Id)  
    ON DELETE RESTRICT  
);  
CREATE INDEX IF NOT EXISTS IX_Bookings_CarId_StartDate_EndDate  
  ON Bookings(CarId, StartDate, EndDate);
```

Main database queries and what they do

These are the key SQLite operations or their SQL equivalents. Most CRUD statements are generated by EF Core from C#.

OPERATION	REPRESENTATIVE SQL	USE
List fleet	SELECT * FROM Cars ORDER BY Id;	Show fleet in stable order.
Get one car	SELECT * FROM Cars WHERE Id = ?;	Find by primary key.
Check date overlap	SELECT EXISTS (SELECT 1 FROM Bookings WHERE CarId = ? AND Status = 'Confirmed' AND StartDate <= ? AND EndDate >= ?);	Reject overlapping confirmed bookings.
Insert reservation	INSERT INTO Bookings (CarId, UserId, StartDate, EndDate, TotalPrice, Status, CreatedAtUtc) VALUES (?, ?, ?, ?, ?, 'Confirmed', ?);	Save valid booking (EF Core SaveChanges).
My bookings	SELECT ... FROM Bookings WHERE UserId = ? ORDER BY CreatedAtUtc DESC;	Only show signed-in member's records.
Cancel reservation	UPDATE Bookings SET Status = 'Cancelled' WHERE Id = ?;	Record cancellation without deleting history.

The overlap check is `existing.StartDate ≤ requested.EndDate AND existing.EndDate ≥ requested.StartDate`. “?” means a parameter supplied safely by EF Core, not string-concatenated user input.

CarRental

A clear foundation for fleet operations

FLEET • BOOKINGS • IDENTITY • SQLITE

Presented by Karan Nale, Om Bangar, Sanket Narsale, Nishant Khatri, and Chirag Dayal.